CSED233 · 데이터구조 · 포스텍

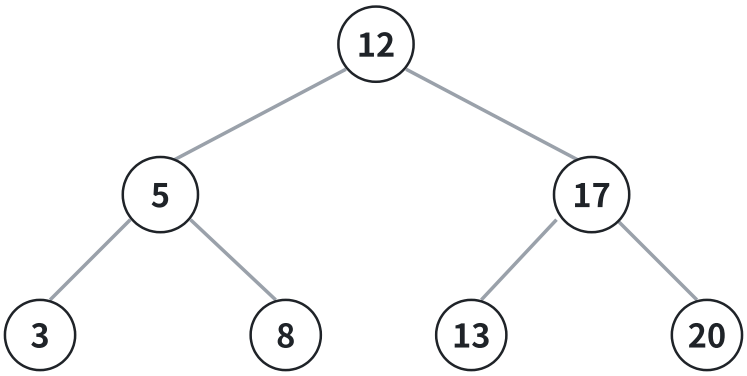
정리본 9. Binary Search Tree

BST 성질 · Search/Insert/Delete · 균형이 성능을 결정한다.

1. BST의 정의와 성질

Binary Search Tree(이진 탐색 트리): 모든 노드에서 왼쪽 서브트리의 모든 값 < 노드 값 < 오른쪽 서브트리의 모든 값을 만족하는 이진 트리.

"직계 자식"이 아니라 "서브트리 전체"와의 관계다 — 이 차이가 시험에 나온다.



검증: 5 왼쪽 {3} < 5 ✓ · 5 오른쪽 {8} > 5 ✓
12 왼쪽 전체 {3,5,8} < 12 ✓ · 12 오른쪽 전체 {13,17,20} > 12 ✓

성질을 만족하는 BST 예시

BST의 존재 이유

Inorder Traversal(중위 순회) 결과 = 오름차순 정렬이다. 위 트리의 inorder는 `3, 5, 8, 12, 13, 17, 20`.

- 별도 정렬 알고리즘 없이 정렬된 데이터를 얻는다.
- 삽입·삭제가 일어나도 정렬 상태가 유지되는 *dynamic sorted data* 구조.

2. 기본 연산

2.1 Search(탐색)

Search(8): 12에서 시작 → 8 < 12 왼쪽 → 8 > 5 오른쪽 → 8 발견. 경로 `12 → 5 → 8`, 3번 비교.
시간복잡도 = **O(높이)**.

높이가 전부다

높이는 간선 수 기준 (정리본 5와 동일 — Leaf=0). 탐색을 포함한 삭제 연산 전체는 O(h)다.

- 균형 BST: $O(\log n)$ — 1,000,000개에서 약 20번 비교.
- 편향 BST: $O(n)$ — 최악 1,000,000번 비교, 사실상 연결 리스트.

2.2 Insert(삽입)

① Search와 같은 방식으로 내려가며 **빈 자리를 찾는다**.

② 찾은 리프 위치에 새 노드를 단다.

```
Insert(4): 5 → 왼쪽 3 → 3의 오른쪽이 비어 있음 → 거기에 4

Before:  5      After:  5
        / \             / \
       3   8           3   8
                        \
                         4
```

시간복잡도: O(높이) — 균형이면 $O(\log n)$.

2.3 Delete(삭제) — 3가지 경우

Case 1 · Leaf

자식 없음 → 그냥 제거. 비용 (찾은 뒤) $O(1)$.

```
Delete(3):
5       5
```

```
/\   →   \
3 8       8
```

Case 2 · 자식 1개

자식이 자리를 물려받는다. 비용 (찾은 뒤) $O(1)$.

```
Delete(5):
 12        12
/     →    /
5          3
/
3
```

Case 3 · 자식 2개 — 핵심

Successor(오른쪽 서브트리의 최솟값)로 값을 대체한 뒤, successor를 삭제한다.

successor는 자식이 최대 1개라 Case 1/2로 환원된다.

예제 · Delete(12) — 자식 2개

① 오른쪽 서브트리(17)에서 계속 왼쪽으로 내려가 최솟값 **13**을 찾는다.

② 12의 값을 13으로 대체한다.

③ 원래 13 노드를 Case 1/2 방식으로 삭제한다.

```
Before:      12          After:       13
          /    \                    /    \
         5     17                  5     17
        /\    /  \                /\      \
       3  8 13    20              3  8      20
```

왜 successor인가

Successor는 삭제 노드보다는 크고 오른쪽 서브트리의 다른 모든 노드보다는 작다 — 그래서 대체해도 **BST 성질이 자동 유지**된다. (왼쪽 서브트리의 최댓값, predecessor를 써도 같은 논리로 된다.)

3. 균형 vs 편향 — 삽입 순서가 운명을 가른다



같은 데이터라도 삽입 순서에 따라 균형/편향이 갈린다

삽입 순서

- `[1, 2, 3, 4, 5]` 순으로 삽입 → 높이 4의 편향 트리, 성능 $O(n)$.
- `[3, 1, 5, 2, 4]` 순으로 삽입 → 높이 2의 균형 트리, 성능 $O(\log n)$.

삽입 순서를 통제할 수 없다면 스스로 균형을 잡는 트리가 필요하다 → 정리본 10의 **AVL Tree**, Red-Black Tree.

4. Quiz

Q1. BST에서 Inorder 순회가 중요한 이유는?

힌트

왼쪽 < 노드 < 오른쪽을 inorder 순서(왼쪽 → 노드 → 오른쪽)로 방문하면 어떤 순서가 나올까.

정답

개념은 힌트 참고.

정답: Inorder 결과가 오름차순 정렬이기 때문. 별도 정렬 없이 정렬된 데이터를 얻을 수 있다.

Q2. Degree-2 노드 삭제 시 왜 Successor를 사용하는가?

힌트

Successor는 오른쪽 서브트리의 최솟값 — 크기 관계를 생각해 보자.

> **정답**

개념은 힌트 참고.

정답: Successor는 삭제 노드보다 크면서 오른쪽 서브트리의 모든 노드보다 작으므로, 대체해도 BST 성질이 자동으로 유지되기 때문. 게다가 자식이 최대 1개라 삭제가 간단하다.

Q3. [1, 2, 3, 4, 5] 순서로 Insert한 후 트리의 높이는?

> **힌트**

매번 루트보다 큰 값이 들어온다. 새 노드는 항상 어디에 붙을까.

> **정답**

개념은 힌트 참고.

정답: 높이 4 (간선 기준, 노드 수 기준이면 5) — 모든 노드가 오른쪽으로만 이어진 편향 트리. 성능이 $O(n)$까지 저하되므로 자동 균형 트리가 필요하다.

Q4. BST 성능이 $O(n)$까지 나빠지는 이유는?

> **힌트**

모든 연산의 비용은 경로 길이 = 높이에 비례한다.

> **정답**

개념은 힌트 참고.

정답: 편향된 순서로 삽입되면 높이가 n까지 커져 선형 탐색과 다를 바 없어진다. AVL/Red-Black Tree 같은 자동 균형 트리로 해결한다.

Q5. Search / Insert / Delete의 시간복잡도가 동일한 이유는?

> **힌트**

세 연산 모두 공통적으로 하는 일이 있다.

정답

개념은 힌트 참고.

정답: 모두 경로상의 노드만 방문하고, 경로 길이 = 트리 높이라서. 균형 BST에서는 셋 다 $O(\log n)$.